# 1 FreeRTOS

**FreeRTOS** is a small real-time kernel: tasks, a scheduler driven by a periodic tick, delays, and tools for tasks that share data. It is written in C, with a few lines of assembly for each CPU family. It is used in industry, in millions of products (sensors, cars, home devices), and its MIT licence lets anyone use and modify it.

FreeRTOS provides ready-made the same mechanisms built by hand in ScratchOS. The kernel is used as it is and never modified: the application consists of the tasks written on top of it.

## 1.1 Same Board, Same Mechanisms

FreeRTOS runs on the same `mps2-an385` board and in the same QEMU as ScratchOS. The boot, the vector table, SysTick, and PendSV work exactly as described in the previous chapters.

The part of the kernel that depends on the CPU is called the **port**. The Cortex-M3 uses the `ARM_CM3` port ( `FreeRTOS/Source/portable/GCC/ARM_CM3/` ), which contains the exception handlers and the context switch; the rest of the kernel ( `tasks.c` , `list.c` , `queue.c` ) is portable C.

| ScratchOS | FreeRTOS ( `ARM_CM3` port) |
|---|---|
| `struct tcb` , saved SP first | `TCB_t` , `pxTopOfStack` first |
| `task_init_stack` | `pxPortInitialiseStack` |
| `PendSV_Handler` | `xPortPendSVHandler` |
| `SysTick_Handler` | `xPortSysTickHandler` → `xTaskIncrementTick` |
| Start of the first task through `svc` | `vPortSVCHandler` , `prvPortStartFirstTask` |
| `task_switch_context` | `vTaskSwitchContext` |
| `idle_task` | `prvIdleTask` |

## 1.2 What Changes

Next to the familiar mechanisms, FreeRTOS introduces a few new ideas.

| Aspect | ScratchOS | FreeRTOS |
|---|---|---|
| Ready tasks | One ready queue, scanned round-robin | One ready list per priority ( `pxReadyTasksLists[]` ) |
| Scheduling rule | All tasks equally important | The highest-priority READY task runs; equal priorities take turns |
| Task state | A `state` field in the TCB | The list the task is in (ready, delayed, suspended) |
| Task memory | Static partitions in the linker script | TCB and stack taken from a heap ( `heap_4.c` ) by `xTaskCreate` |
| Critical sections | `PRIMASK` : all interrupts masked | `BASEPRI` : only interrupts up to a given priority masked |

| Aspect | ScratchOS | FreeRTOS |
|---|---|---|
| Task privilege | Unprivileged, confined by the MPU | Privileged, no memory protection between tasks |
| End of a task | Returning leads to a clean exit | A task function must never return |

> ⚠ **TWO OPPOSITE PRIORITY SCALES IN FREERTOS A HIGHER TASK PRIORITY IS MORE URGENT (0 IS THE IDLE TASK). IN THE CORTEX-M3 NVIC AND SCB A LOWER PRIORITY VALUE IS MORE URGENT. THE TWO SCALES MUST NOT BE CONFUSED.**

## 1.3 Kernel Configuration: `FreeRTOSConfig.h`

The kernel reads `FreeRTOSConfig.h` to decide which features to build and how large its tables are.

```
#define configUSE_PREEMPTION            1      /* a more urgent READY task takes the CPU at
once */
#define configUSE_TIME_SLICING          1      /* equal priorities take turns, one tick each */
#define configMAX_PRIORITIES            ( 5 )  /* priorities 0 (idle task) to 4; higher = more
urgent */
#define configCPU_CLOCK_HZ              ( ( unsigned long ) 25000000 )   /* 25 MHz */
#define configTICK_RATE_HZ              ( ( TickType_t ) 100 )           /* 100 ticks per
second */
#define configMINIMAL_STACK_SIZE        ( ( unsigned short ) 256 )       /* in WORDS */
#define configTOTAL_HEAP_SIZE           ( ( size_t ) ( 64 * 1024 ) )     /* bytes */
#define configCHECK_FOR_STACK_OVERFLOW  2      /* check the stack at every switch */
```

With these values one tick lasts 1/100 s = **10 ms**, the same tick as ScratchOS, and a task created with `configMINIMAL_STACK_SIZE` gets 256 words × 4 bytes = **1 KB** of stack.